SVEUČILIŠTE U ZAGREBU

FAKULTET ORGANIZACIJE I INFORMATIKE

V A R A Ž D I N

Mateo Zović

FENWICKOVO STABLO

SEMINARSKI RAD

Varaždin, 2023.

SVEUČILIŠTE U ZAGREBU

FAKULTET ORGANIZACIJE I INFORMATIKE

V A R A Ž D I N

Mateo Zović

Matični broj: 49277/21-R

Studij: Informacijski i poslovni sustavi

FENWICKOVO STABLO

SEMINARSKI RAD

Mentor:

Prof. dr. sc. Alen Lovrenčić

Varaždin, siječanj 2023.

Sadržaj

1. Uvod 1
2. Fenwickovo stablo 2
2.1. Prikaz pomoću polja 3
2.2. Prikaz pomoću stabla 5
3. Popunjavanje i ažuriranje stabla 7
3.1. Popunjavanje stabla 7
3.2. Ažuriranje stabla 10
4. Izračun sume elemenata 12
5. Implementacija Fenwickovog stabla 14
6. Složenost operacija 17
7. Amortizirana složenost 21
8. Zaključak 23
Popis literature 24

1. Uvod

Struktura podataka predstavlja način pohrane podataka u obliku nekog programskom koda kako bi se kasnije taj kod iskoristio za efikasnu uporabu u drugim programskim rješenjima. Pri izradi složenijih programskih rješenja jedan od najvažnijih koraka jest odabir strukture podataka koja će se primijeniti. Najčešće je odabirom strukture podataka određena i vremenska složenost cijelog koda. Strukture podataka se mogu podijeliti na statičke i dinamičke strukture. Statičke strukture su one kojima se broj elemenata prilikom izvođenja programa ne mijenja, to jest fiksne su veličine, dok se kod dinamičkih mijenja prema potrebama. Prednost dinamičke strukture u odnosu na statičku je ta da ona zauzima točno onoliko memorije koliko je potrebno za izvršavanje programa. Primjer statičke strukture bili bi nizovi jer se njihova veličina zadaje prilikom same deklaracije, a primjer dinamičke strukture su liste, stabla, grafovi i tako dalje. Ovaj seminarski rad obrađuje upravo dinamičku strukturu podataka i to stablo, konkretno Fenwickovo stablo. Fenwickovo stablo je struktura podataka, koja uvodi optimalno rješenje za izračun sume prefiksa. Potječe iz 1989. godine kada ju je Boris Ryabko osmislio, te modificirao 1992. Struktura stabla omogućuje izvođenje operacija dohvaćanja elemenata, ažuriranja te zbroja prefiksa i zbroja raspona elemenata.

2. Fenwickovo stablo

Fenwickovo stablo je struktura podataka zapisana najčešće u nizu, koja izračunava sumu brojeva u zadanom intervalu. Ti brojevi mogu biti zapisani u nekom polju, vektoru i slično, a obilježeni su indeksom. Ovo stablo, ili bolje rečeno struktura podataka, još je poznata i pod imenom binarno indeksirano stablo. Fenwickovo stablo je karakteristično po tome što svaki element pamti sumu brojeva iz nekog intervala. Interval ovisi o indeksu elementa, koji se interpretira pomoću binarnog zapisa. Slijedi primjer.

Imamo polje P od 11 elemenata koje se kreće od indeksa 0 do indeksa 10. U njega su zapisane nasumične vrijednosti.

Indeks i	0	1	2	3	4	5	6	7	8	9	10
Polje P	12	15	23	35	54	10	6	20	38	1	9
Polje F	12	27	23	85	54	64	6	175	38	39	9

U polju F (Fenwickov zapis) se nalaze ti isti brojevi izraženi sumom prethodnih u nekom intervalu. Drugim riječima, određena polja sadrže sumu prefiksa. Suma prefiksa je zbroj svih brojeva u nizu od početka do nekog zadanog indeksa. Može se uočiti da se u zapisu polja F svaki drugi broj prepisuje (indeksi 0, 2, 4, 6, 8, 10), dok se u preostalim brojevima računa suma. Naizgled možda djeluje nerazumno i besmisleno, međutim, ova metoda zapisa elemenata u polje uvelike ubrzava izračun sume brojeva u željenom intervalu i smanjuje vremensko izvođenje programa. To je uvelike vidljivo u programskim rješenjima gdje je potrebno izračunati sumu mnoštva elemenata.

2.1. Prikaz pomoću polja

Radi jednostavnosti i lakšeg razumijevanja prvo će biti prikazan zapis elemenata u polje, a potom zapis u stablo. Da bi razumjeli kako se brojevi zapisuju u polje F potrebno je prisjetiti se zapisa prirodnih brojeva u binarnome obliku. Binarni zapis indeksa – pozicija na kojoj se element nalazi u polju, govori o tome koja suma brojeva će se zapisati na pojedino mjesto u polju. Bitno je napomenuti da se u praksi element s indeksom 0 ne koristi jer on kod implementacije stabla predstavlja korijen stabla. Stoga se prvi broj pohranjuje na indeks 1.

Napomena: Prikaz pomoću polja služi radi naglaska na binarnom zapisu indeksa te radi jednostavnosti prikaza izračuna vrijednosti elemenata. Fenwickovo stablo se zapisuje u strukturu stabla koja je nalik strukturi polja, tako da ovaj dio je samo pomoć i priprema za lakše razumijevanje zapisa u stablo, koje slijedi nakon ovog poglavlja.

Indeks	Binarni zapis indeksa	Polje P	Polje F
1	0001	12	
2	0010	15	
3	0011	23	
4	0100	35	
5	0101	54	
6	0110	10	
7	0111	6	
8	1000	20	
9	1001	38	
10	1010	1	
11	1011	9	

Kako bi se dobio zapis u polju F, u prvom koraku se gledaju oni binarni zapisi u kojima se pojavljuje vrijednost 1 na mjestu prvog bita s desne strane. Kod tih elemenata u polju, vrijednost elementa se prepiše u polje F.

Indeks	Binarni zapis indeksa	Polje P	Polje F
1	000**1**	12	**12**
2	0010	15	
3	001**1**	23	**23**

4	0100	35	
5	010**1**	54	**54**
6	0110	10	
7	011**1**	6	**6**
8	1000	20	
9	100**1**	38	**38**
10	1010	1	
11	101**1**	9	**9**

U drugom koraku gledamo koji binarni zapis sadrži vrijednost 1 na drugome mjestu. Kod takvih elemenata zbrajamo vrijednost elementa prošlog s vrijednošću trenutnog te se rezultat zapisuje u polje F. Gledaju se samo elementi koji su na prethodnom mjestu imali bit 0.

Indeks	Binarni zapis indeksa	Polje P	Polje F
1	0001	12	12
2	00**1**0	15	**27**
3	0011	23	23
4	0100	35	
5	0101	54	54
6	01**1**0	10	**64**
7	0111	6	6
8	1000	20	
9	1001	38	38
10	10**1**0	1	**39**
11	1011	9	9

Ovaj korak se ponavlja iznova sve dok se ne popuni polje F. U trećem prolazu se gleda treći bit binarnog zapisa, uz uvjet da su prethodna dva bita 0. U četvrtom prolazu prvi bit mora biti 1, dok svi ostali 0. Zapis u polje F je zbroj trenutnog elementa i prethodna 3 elementa, ako se radi o trećem prolazu ili prethodnih 7 elemenata ako se radi o četvrtom prolazu. Dakle, suma brojeva ovisi o prolazu, a može se izraziti kao suma 2^{n-1} elemenata, gdje n označava broj prolaza. Broj prolaza označava poziciju bita u binarnom zapisu gledajući s desna prema lijevo. Prolazi su slikovito prikazani kako bi se uvidio postupak dobivanja sume brojeva.

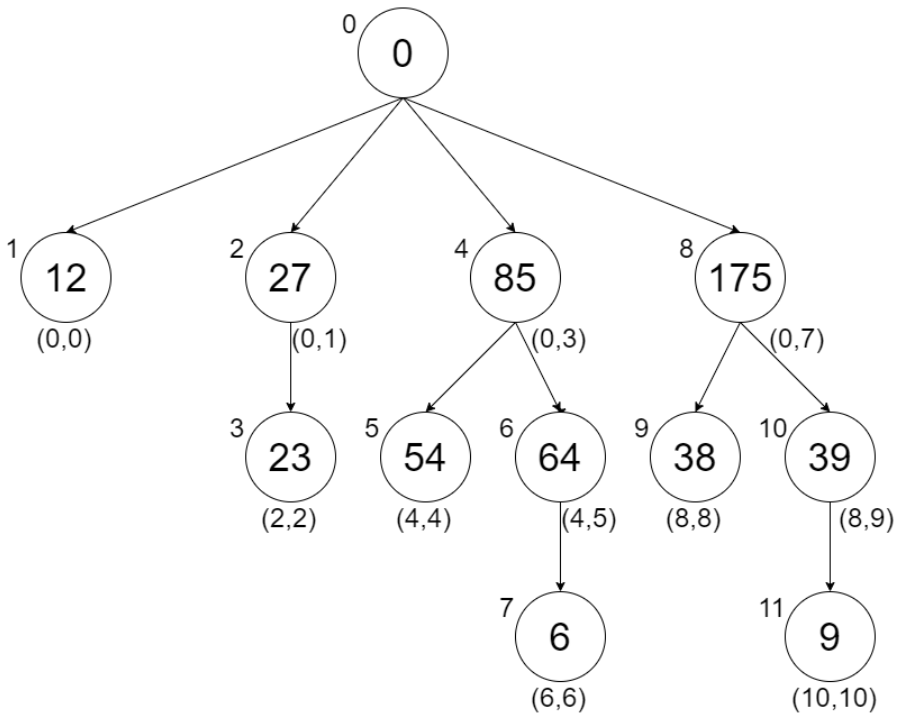
Indeks	Binarni zapis indeksa	Polje P	Polje F
1	0001	12	12
2	0010	15	27
3	0011	23	23
4	0**1**00	35	**85**
5	0101	54	54
6	0110	10	64
7	0111	6	6
8	**1**000	20	**175**
9	1001	38	38
10	1010	1	39
11	1011	9	9

U tablici se na indeksu 4 zbrajaju vrijednosti 12+15+23+35 što daje 85. Na indeksu 8 zbraja se 8 elemenata i to trenutni sa prethodnih 7 elemenata, to jest 12+15+23+35+54+10+6+20 i dobiva se rezultat 175.

2.2. Prikaz pomoću stabla

Vrijednosti koje se upisuju u stablo jednake su onima u polju, a razlika je u tome što stablo ima poseban redoslijed elemenata, dok polje može biti napisano samo u nizu. Kao i kod polja, element u stablu s indeksom 0 se ne koristi za pohranu vrijednosti.

Indeks i	0	1	2	3	4	5	6	7	8	9	10
Polje P	12	15	23	35	54	10	6	20	38	1	9

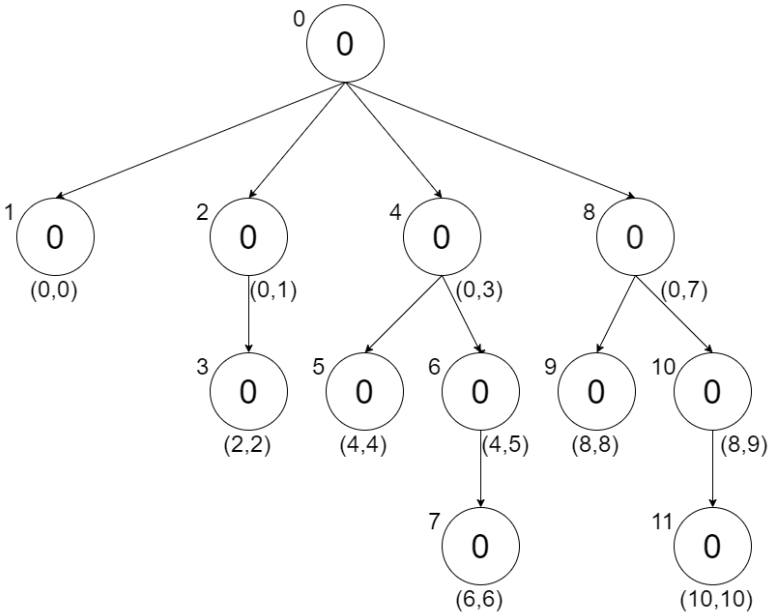


Na gornjoj slici je prikazan primjer Fenwickovog stabla. Prolaz kroz stablo se odvija s lijeva na desno i to prvo u dubinu, a onda u širinu. Broj koji se nalazi u krugu predstavlja vrijednost elementa, u lijevom gornjem kutu pored kruga se nalazi njegov indeks (indeks stabla), dok ispod kruga piše interval indeksa (indeksi polja). Ideja je da svaki put kada se napravi novo dijete iz korijena, to dijete sadrži sumu svih vrijednosti od prošle djece korijena. Dakle, vrijednost elementa koja je zapisana u dijete s indeksom 4 je zajednički zbroj svih vrijednosti u polju indeksa 0,1,2 i 3. Taj interval je u stablu označen s (0,3) na poziciji u stablu s indeksom 4. Ono što je još primjetno je da svaki element u stablu koji je zadan nad nekim intervalom (a ne nad samim sobom npr. (2,2)) ima obavezno barem jedno dijete, a može ih imati i više. Stoga, djeca u stablu sadrže samo svoju vrijednost, dok roditelji sadrže sumu vrijednosti brojeva. Još je bitno za istaknuti kako brojevi na indeksima 1,2,4,8 u binarnom zapisu glase 0001, 0010, 0100, 1000 te kada bi im se prvi bit jedinice zamijenio s bitom 0 svi oni bi poprimili vrijednost 0, po tome se može zaključiti da su oni djeca roditelja na indeksu 0. Uzmimo na primjer indeks broja 11, njegov binarni zapis je 1011, ako mu se zamijeni prvi bit koji iznosi 1 dobit će se 1010, što je u prijevodu oznaka za broj 10, iz toga slijeda da broj 11 mora biti dijete broja s indeksom 10. Broju 1010 se zamijeni sljedeći bit 1 i dobije se binarni zapis indeksa 8 to jest 1000. Po tome je 10 dijete roditelja 8, a još jednom zamjenom dobije se 0000, što predstavlja da je 8 dijete indeksa 0. Na temelju toga je nastao ovakav prikaz Fenwickovog stabla.

3. Popunjavanje i ažuriranje stabla

Popunjavanje i ažuriranje Fenwickovog stabla će biti prikazano pomoću slika, gdje će se u koracima moći pratiti što se događa s vrijednostima elemenata u stablu. Za početak je prikazano polje P s nasumičnim vrijednostima, kao i prazno stablo (stablo popunjeno s vrijednosti 0).

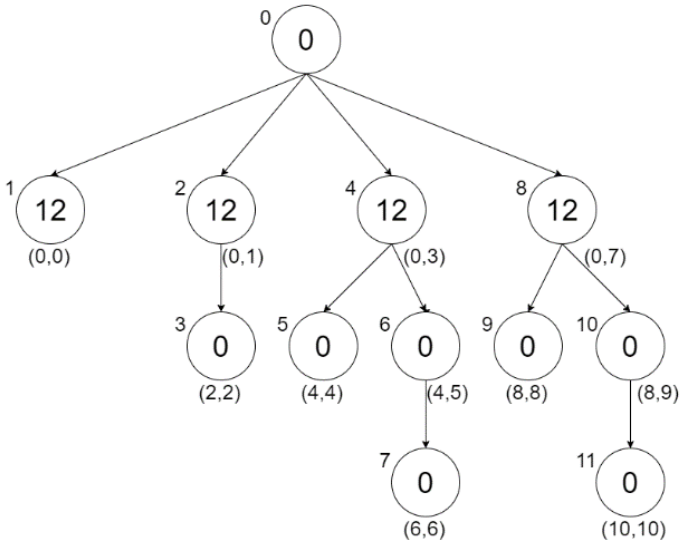
Indeks i	0	1	2	3	4	5	6	7	8	9	10
Polje P	12	15	23	35	54	10	6	20	38	1	9



3.1. Popunjavanje stabla

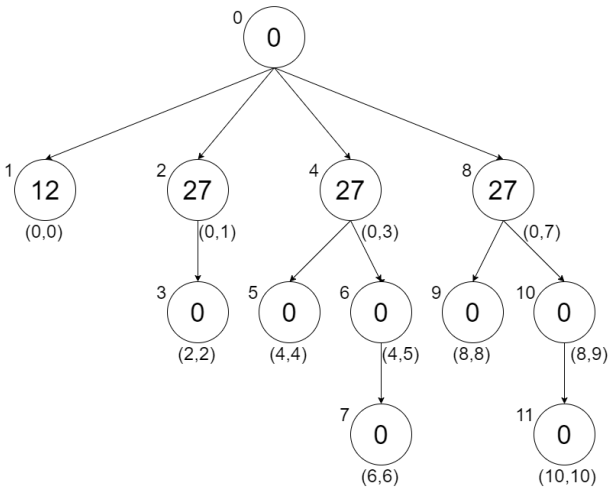
Stablo se popunjuje na način da se broj iz polja zbraja s vrijednosti onog mjesta u stablu gdje je on prisutan kao interval. Na primjer broj 12 je element u polju s indeksom 0. Taj broj će se zbrojiti sa svim pozicijama u stablu gdje je on dio intervala (interval svake pozicije je zapisan ispod kruga). Stoga, broj 12 se upisuje na mjesto s indeksom 1 u stablu (gornji lijevi kut pored kruga označava indeks elementa u stablu), upisuje se i na mjesto 2, 4 i 8. Ispravnije bi bilo reći da se zbraja sa prošlom vrijednosti koja je bila zapisana na tom mjestu (vrijednost 0).

Korak 1 (vrijednost 12)

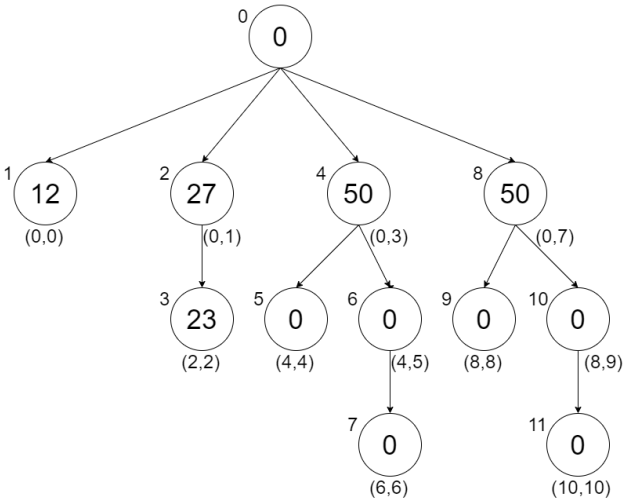


U drugom koraku se dodaje vrijednost 15 u stablo, na isti način kao što se je dodao i 12. Dakle, broj 15 se je pribrojio vrijednosti u stablu na indeksu 2, 4 i 8 jer se nalazi u tome intervalu i dobivena suma iznosi 27. U trećem koraku se vrijednost 23 pohranjuje na mjesto u stablu na indeksu 3, kao što se i zbraja sa vrijednosti na indeksu 4 i 8, što daje 50. Ovi koraci se ponavljaju sve dok se ne upišu svi elementi u stablo, tako da je nepotrebno opisivati što se događa pri svakom koraku.

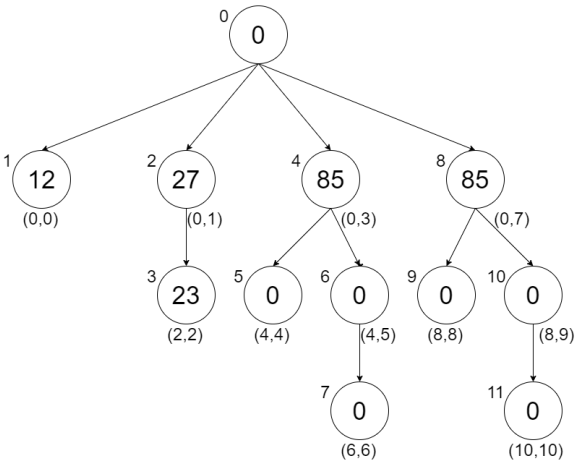
Korak 2 (vrijednost 15)



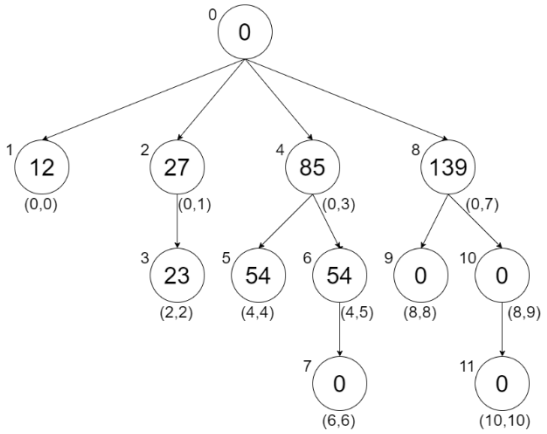
Korak 3 (vrijednost 23)



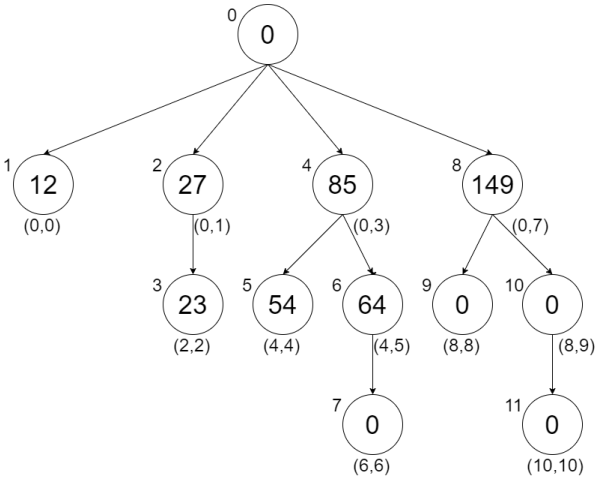
Korak 4 (vrijednost 35)



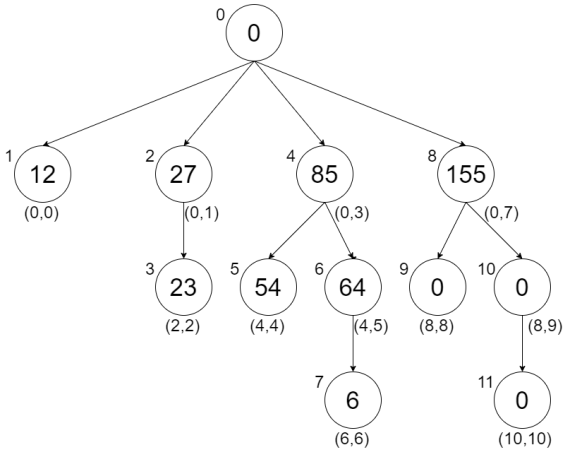
Korak 5 (vrijednost 54)



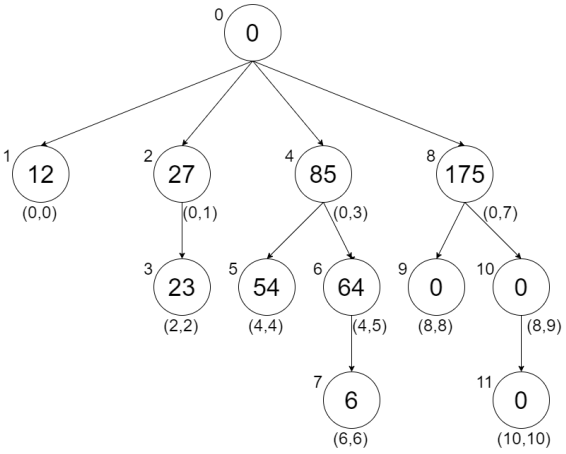
Korak 6 (vrijednost 10)



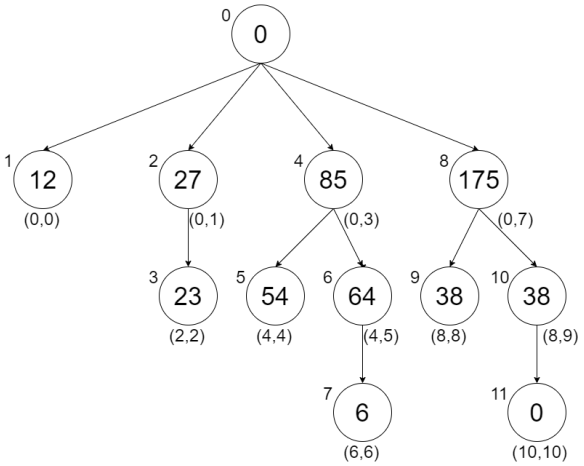
Korak 7 (vrijednost 6)



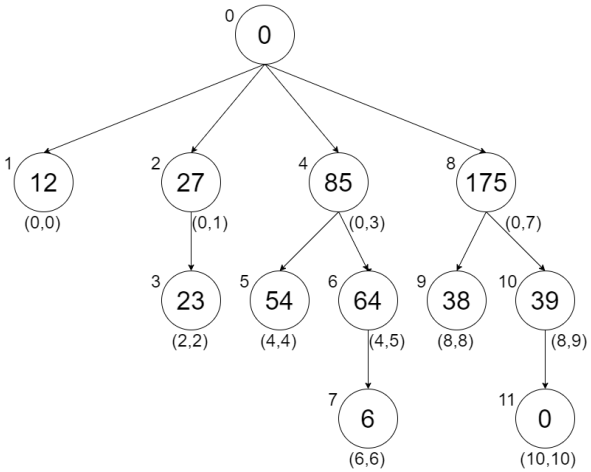
Korak 8 (vrijednost 20)



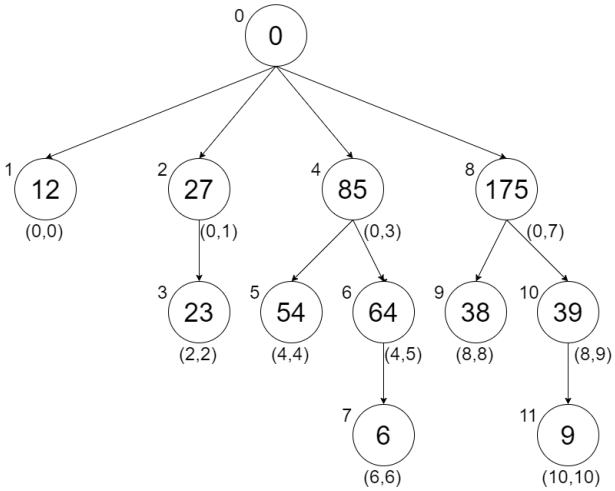
Korak 9 (vrijednost 38)



Korak 10 (vrijednost 1)



Korak 11 (vrijednost 9)



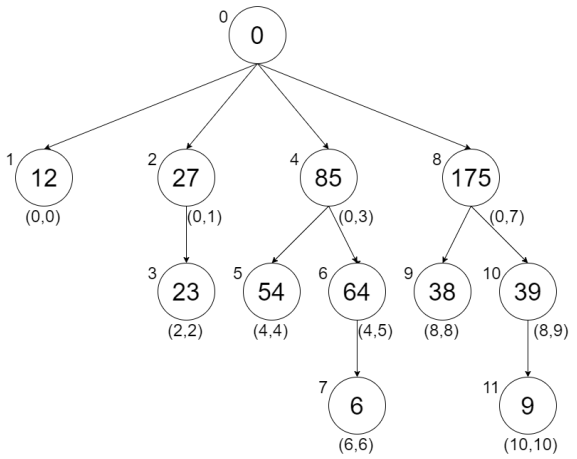
3.2. Ažuriranje stabla

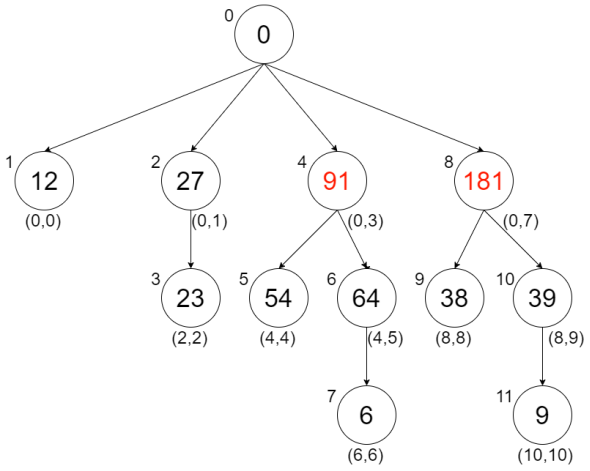
Ažuriranje Fenwickovog stabla se odvija na isti način kao i punjenje stabla. Kada se promijeni vrijednost nekog elementa, tada se mijenjaju i sve vrijednosti u stablu koje sadrže taj element u svome intervalu. Slijedi primjer. Element na indeksu 3 umjesto 35 će iznositi 41.

Indeks i	0	1	2	3	4	5	6	7	8	9	10
Polje P	12	15	23	41	54	10	6	20	38	1	9

Prvo se izračuna kolika je promjena u odnosu na prošlu vrijednost. Dakle, 41-35 i dobije se promjena od 6. Vrijednost 6 se dodaje elementima u stablu s indeksima 4 i 8.

Primjer 1 (dodavanje vrijednosti 6)

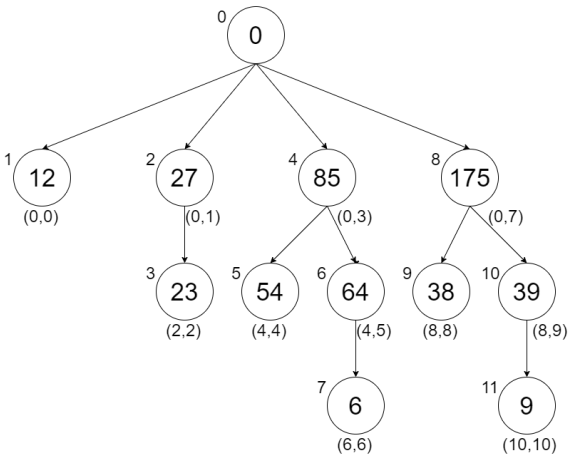


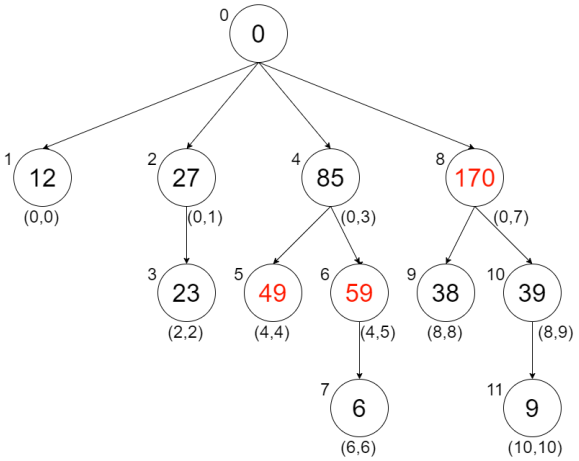


Za drugi primjer uzmimo da je broj na indeksu 4 iznosi 49 umjesto 54. Promjena će iznositi -5, pa će se elementi na indeksima 5, 6 i 8 umanjiti za 5. Zašto se elementi tih indeksa smanjuju, zato što svi oni u svome intervalu sadrže broj s indeksom 4.

Indeks i	0	1	2	3	4	5	6	7	8	9	10
Polje P	12	15	23	35	49	10	6	20	38	1	9

Primjer 2 (dodavanje vrijednosti -5)



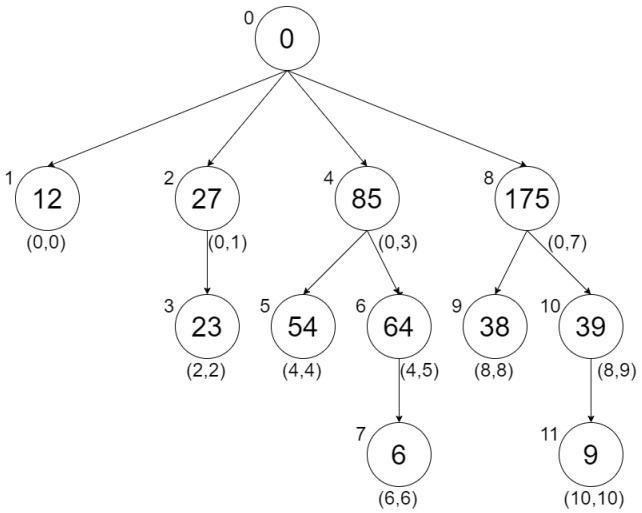


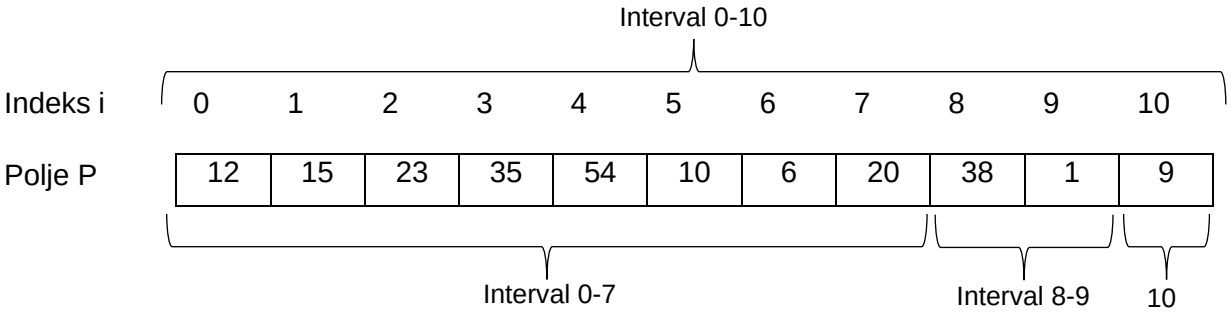
4. Izračun sume elemenata

Glavna funkcija Fenwickovog stabla je izračun sume brojeva u nekom intervalu u što manje koraka. Princip rada će biti prikazan pomoću sljedećih primjera. Za lakše razumijevanje koristit će se prikaz polja i stabla koji su već prethodno bili prikazani.

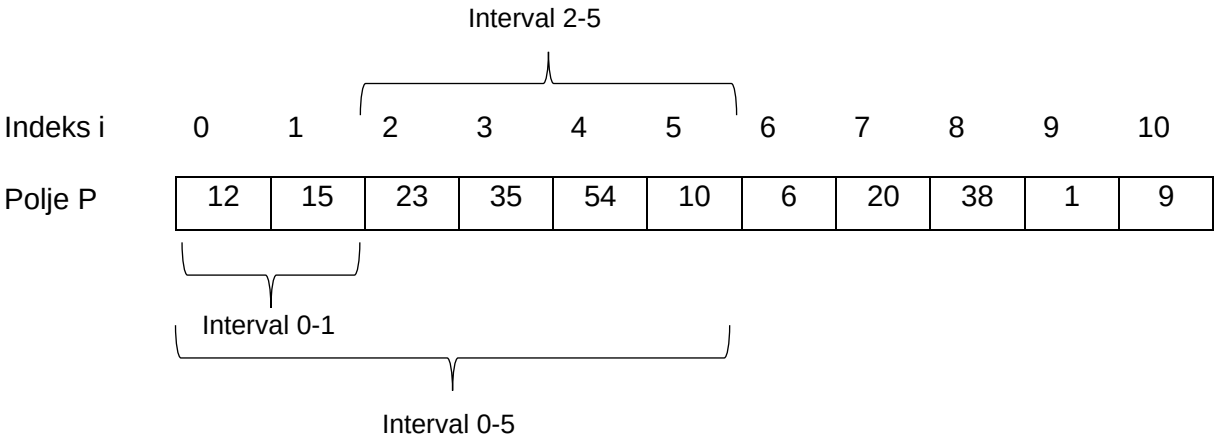
Indeks i	0	1	2	3	4	5	6	7	8	9	10
Polje P	12	15	23	35	54	10	6	20	38	1	9

Traži se suma elemenata u intervalu indeksa 0-5. Zadani interval sadrži 6 brojeva stoga se uzima vrijednost elementa na indeksu 6 te se toj vrijednosti dodaje vrijednost roditelja. Taj proces se ponavlja sve dok se ne stigne do korijena stabla. Dakle, suma brojeva u intervalu 0-5 je 64+85=149. Ovom tehnikom se je smanjilo vrijeme trajanja programa jer program nije zbrajao svih 6 elemenata pojedinačno, već se poslužio zapisima u stablu, gdje je u 2 koraka došao do rezultata. Suma intervala 0-10 bi glasila vrijednost elementa s indeksom 11 je 9, + roditelj 39, + roditelj 175, došao do korijena, stani i rezultat je 223. Grafički se to može prikazati na sljedeći način:





Može se uočiti da je suma brojeva u rasponu od 0 do 10 dobivena kao zbroj intervala elemenata u rasponu 0-7, 8-9 i 10. No što ako se traži suma elemenata koja ne kreće od početka stabla, kao na primjer suma brojeva u intervalu 2-5. Tada se jednostavno izračuna suma prefiksa 0-5, i suma 0-1 te se razlikom te dvije sume dobije zajednički zbroj elemenata u zadanom intervalu. Razlog zašto je drugi interval 0-1, a ne 0-2 je zato što je element na poziciji 2 uključen u sumu elemenata.



Suma elemenata u intervalu 0-5 iznosit će 149. Interval 0-1 daje sumu 27. Oduzimanjem te dvije sume prefiksa dobiva se iznos 122, koji odgovara sumi elemenata u intervalu 2-5.

5. Implementacija Fenwickovog stabla

Programska implementacija Fenwickovog stabla sastoji se od funkcija Dodaj, Ažuriraj, Suma i funkcije Izbriši. Uz te četiri funkcije koristi se i konstruktor koji kreira stablo određene duljine, te destruktor koji služi za brisanje elemenata stabla.

```
1   #include <iostream>
2
3   template <typename nodetype>
4   class Fenwick{
5       private:
6           nodetype *stablo;
7           int n;
8
9       public:
10      Fenwick(int duljina):n(duljina){
11          stablo=new nodetype[n+1];
12      }
13
14      void Dodaj(int x, nodetype vrijednost){
15          while(x<=n){
16              stablo[x] += vrijednost;
17              x += x&-x;
18          }
19      }
20
21      void Azuriraj(int x, nodetype vrijednost){
22          nodetype promjena=vrijednost-stablo[x];
23          Dodaj(x, promjena);
24      }
25
26      nodetype Suma(int x){
27          nodetype zbroj=0;
28          while(x>0){
29              zbroj += stablo[x];
30              x -= x&-x;
31          }
32          return zbroj;
33      }
34
35      void Izbrisi(int x){
36          nodetype promjena=-stablo[x];
37          Dodaj(x, promjena);
38      }
39
40      ~Fenwick() {
41          delete[] stablo;
42      }
43  };
```

Prethodna slika prikazuje programski kod Fenwickovog stabla implementiranog kao klasa template, tako da može raditi s bilo kojom vrstom podataka. Klasa ima privatni podatkovni član n, koji pohranjuje veličinu Fenwickovog stabla i privatni podatkovni član stablo, koji je pokazivač na niz koji pohranjuje vrijednosti Fenwickovog stabla. Parametar nodetype koristi se za određivanje tipa podataka vrijednosti pohranjenih u stablu. Klasa uz konstruktor i destruktor sadrži četiri funkcije: Dodaj, Ažuriraj, Suma i Izbriši.

Konstruktor klase uzima cjelobrojni parametar duljina, koji se koristi za inicijalizaciju člana podataka n i dodjeljuje mu memoriju za niz stablo. Također, koristi operator new za dinamičku alokaciju memorije za niz tipa nodetype veličine n+1.

```
Fenwick(int duljina):n(duljina){
    stablo=new nodetype[n+1];
}
```

Funkcija dodavanja omogućuje dodavanje novog elementa u stablo. Prvi argument funkcije označava indeks elementa u koji se upisuje vrijednost, dok drugi argument predstavlja vrijednost koja se upisuje. U početku je vrijednost svakog elementa 0, pa se dana vrijednost samo zbroji s vrijednosti u polju, međutim, pri ažuriranju elemenata se koristi posebna funkcija.

```
void Dodaj(int x, nodetype vrijednost){
    while(x<=n){
        stablo[x] += vrijednost;
        x += x&-x;
    }
}
```

Kod ažuriranja se prije pozivanja prethodne funkcije Dodaj mora izračunati promjena u odnosu na vrijednost elementa koji je zapisan u stablo. U funkciji ažuriranja se računa promjena između novog elementa i već postojećeg elementa u stablu. Kada se dobije vrijednost promjene, poziva se funkcija Dodaj s argumentima x i promjenom, to jest novom vrijednosti koja se dodaje elementu stabla na poziciji s indeksom x.

```
void Azuriraj(int x, nodetype vrijednost){
    nodetype promjena=vrijednost-stablo[x];
    Dodaj(x, promjena);
}
```

Funkcija Suma izračunava zbroj prefiksa u stablu to jest sumu vrijednosti do određenog indeksa x. Započinje od indeksa x i ponavlja se sve dok ne dođe do korijena stabla. Pri svakom ponavljanju while petlje, indeks se smanjuje i premješta na roditelja prošlog indeksa. Kada indeks dođe do korijena stabla vraća sumu elemenata.

```
nodetype Suma(int x){
    nodetype zbroj=0;
    while(x>0){
        zbroj += stablo[x];
        x -= x&-x;
    }
    return zbroj;
}
```

Funkcija Izbriši je vrlo slična funkciji Ažuriraj, jedina razlika je u tome što ona prima samo jedan argument i to indeks elementa koji se želi izbrisati. U funkciji se kreira varijabla promjena koja je tipa nodetype te njena vrijednost postaje suprotna vrijednosti elementa na indeksu x u stablu. Zatim se poziva funkcija Dodaj, koja u ovome slučaju vrijednost elementa s indeksom x postavlja na 0.

```
void Izbrisi(int x){
    nodetype promjena=-stablo[x];
    Dodaj(x, promjena);
}
```

Destruktor klase oslobađa dinamičku alociranu memoriju.

```
~Fenwick() {
    delete[] stablo;
}
```

6. Složenost operacija

Vremenska složenost konstruktora:

- Dodjeljuje se parametar duljina varijabli privatnog člana n.
- Stvara se novo polje veličine n+1 i dodjeljuje njegov pokazivač privatnoj varijabli člana stablo.
- Inicijalizira se svaki element niza na 0.

Stvaranje niza od n elemenata i inicijalizacija svakog elementa traje O(n) vremena. Trajanje same dodjele vrijednosti je složenost O(1). Dakle, ukupna vremenska složenost konstruktora je O(n).

```
Fenwick(int duljina):n(duljina){
    stablo=new nodetype[n+1];
}
```

Vremenska složenost funkcije Dodaj:

- Funkcija započinje od indeksa x u Fenwickovu stablu.
- Ažurira vrijednost na tom indeksu dodavanjem zadane vrijednosti.
- Zatim se pomiče na sljedeći indeks dodavanjem najmanje značajnog bita od x. To se postiže korištenjem operacije AND po bitovima, s negacijom x po bitovima (x & -x).
- Ponavljaju se prethodna dva koraka sve dok se ne dosegne indeks veći od n, nakon čega funkcija izlazi.

Broj ponavljanja u zadnjem koraku određen je brojem bitova potrebnih za predstavljanje indeksa x. Budući da je veličina Fenwickovog stabla n, maksimalni broj bitova potrebnih za predstavljanje indeksa je log2(n). Stoga je vremenska složenost funkcije Dodaj O(log n).

```
void Dodaj(int x, nodetype vrijednost){
    while(x<=n){
        stablo[x] += vrijednost;
        x += x&-x;
    }
}
```

Vremenska složenost funkcije Ažuriraj:

- Izračunava se razlika između nove vrijednosti (vrijednost) i trenutne vrijednosti na indeksu x u Fenwickovu stablu (stablo[x]).
- Poziva se funkcija Dodaj za ažuriranje vrijednosti indeksa x i svih pogođenih indeksa prosljeđivanjem izračunate razlike.
- Funkcija Dodaj ima vremensku složenost od O(log n) kao što je već prethodno navedeno.

Budući da funkcija Ažuriraj poziva samo funkciju Dodaj, njena vremenska složenost je također O(log n) što je vremenska složenost funkcije Dodaj.

```
void Azuriraj(int x, nodetype vrijednost){
    nodetype promjena=vrijednost-stablo[x];
    Dodaj(x, promjena);
}
```

Vremenska složenost funkcije Suma:

- Započinje od indeksa x u Fenwickovu stablu.
- Zbraja vrijednost tog indeksa u posebnu varijablu zbroj.
- Zatim se pomiče na sljedeći indeks oduzimanjem najmanjeg bita od x. To se postiže korištenjem operacije AND po bitovima, s negacijom x po bitovima (x & -x).
- Ponavljaju se prethodna dva koraka sve dok se ne dosegne indeks 0 (korijen stabla), nakon čega funkcija izlazi.

Broj ponavljanja u zadnjem koraku određen je brojem bitova potrebnih za predstavljanje indeksa x. Budući da je veličina Fenwickovog stabla n, maksimalni broj bitova potrebnih za predstavljanje indeksa je log2(n). Stoga je vremenska složenost funkcije Suma O(log n).

```
nodetype Suma(int x){
    nodetype zbroj=0;
    while(x>0){
        zbroj += stablo[x];
        x -= x&-x;
    }
    return zbroj;
}
```

Vremenska složenost funkcije Izbriši:

- Unutar funkcije se izračunava negativna vrijednost trenutne vrijednosti na indeksu x u Fenwickovom stablu (stablo[x]) te se ta vrijednost pohranjuje u varijablu promjena.
- Poziva se funkcija Dodaj za ažuriranje vrijednosti indeksa x i svih pogođenih indeksa prosljeđivanjem varijable promjena.
- Funkcija Dodaj ima vremensku složenost O(log n).

Budući da funkcija Izbriši poziva funkciju Dodaj, njena vremenska složenost je O(log n).

U ovom slučaju funkcija Izbriši koristi isti algoritam kao Dodaj, ali s tom razlikom da oduzima vrijednost na svakom indeksu umjesto zbrajanja. Budući da algoritam ostaje isti, također ostaje ista vremenska složenost od O(log n).

```
void Izbrisi(int x){
    nodetype promjena=-stablo[x];
    Dodaj(x, promjena);
}
```

Vremenska složenost destruktora:

- Dealocira se memorija niza na koji ukazuje varijabla privatnog člana stablo pozivanjem operatora delete[].
- Također funkcija oslobađa sve druge resurse koje je Fenwickov objekt možda stekao tijekom svog životnog vijeka.

Oslobađanje memorije niza traje konstantno vrijeme O(1), bez obzira na veličinu niza. Oslobađanje bilo kojih drugih resursa također traje O(1) vremena. Stoga je ukupna vremenska složenost destruktora O(1).

```
~Fenwick() {
    delete[] stablo;
}
```

Dodatno pojašnjenje:

Funkcije Dodaj, Suma, Ažuriraj i Izbriši prolaze kroz Fenwickovo stablo počevši od zadanog indeksa i pomičući se gore ili dolje po stablu manipuliraju najmanje značajnim bitom indeksa. Visina Fenwickovog stabla je log2(n), gdje je n veličina izvornog niza. Stoga je maksimalan broj operacija potrebnih za obilazak stabla log2(n). Budući da je vremenska složenost svake operacije konstantna (O(1)), ukupna vremenska složenost funkcija Dodaj, Suma, Ažuriraj i Izbriši je O(log n). U najgorem slučaju, svaka funkcija treba prijeći stablo od lisnog čvora do korijenskog čvora, tako da se mora pomaknuti prema gore na razine log2(n), a kao što je već poznato, svaka razina ima konstantan broj čvorova. Stoga je ukupan broj operacija potrebnih za obilazak stabla log2(n) * O(1) = O(log n).

7. Amortizirana složenost

Kako bi izračunali amortiziranu vremensku složenost koda, potrebno je uzeti u obzir prosječno vrijeme potrebno za svaku operaciju u nizu operacija. Amortizirana vremenska složenost funkcije Dodaj je O(log n) jer u najgorem slučaju ona mora prijeći stablo od lisnog čvora do korijenskog čvora, a za svaku operaciju je potrebno O(log n) vremena. Amortizirana vremenska složenost funkcije Suma također je O(log n) jer ona također, u najgorem slučaju, mora proći stablo od lista do korijena, a za svaku operaciju potrebno je O(log n) vremena. Amortizirana vremenska složenost funkcije Ažuriraj i Izbriši je O(log n) jer pozivaju funkciju Dodaj koja ima amortiziranu vremensku složenost od O(log n).

Amortizirana vremenska složenost svih operacija u Fenwickovoj klasi stabla je O(log n) jer je prosječno potrebno O(log n) vremena za izvođenje svake operacije u nizu operacija. U sigma zapisu bi to izgledalo ovako:

$$O(\Sigma(\log n)) = O(\log n + \log n + \log n + \log n) = O(4 \log n) = O(\log n)$$

Ovaj zapis predstavlja ukupno vrijeme potrebno za sve operacije u nizu operacija, također može se primijetiti kako su konstante u izračunu zanemarene radi preglednijeg ispisa. Konstante ne mijenjaju amortiziranu složenost i iz tog razloga nisu prikazane. Klasa stabla Fenwick ima 4 glavne funkcije (Dodaj, Suma, Ažuriraj, Izbriši) i svaka funkcija ima amortiziranu vremensku složenost od O(log n). Iz tog razloga se sve one zbrajaju i dobiva se O(4 log n) to jest O(log n), gdje n predstavlja veličinu izvornog niza. Zaključno, amortizirana vremenska složenost svih operacija u Fenwickovoj klasi stabla je O(log n) i ona predstavlja prosječnu vremensku složenost svih operacija.

Stvarna cijena operacija	
Operacija	Cijena
Dodaj	log i
Ažuriraj	log i
Suma	log i
Izbriši	log i

U ovoj implementaciji nema operacija koje mijenjaju potencijal strukture podataka. Kandidati za mijenjanje potencijala strukture bile bi operacije Dodaj i Izbriši, međutim nisu. Razlog tome je što se pri korištenju Fenwickovog stabla, u početku, mora odabrati veličina polja u koje će se pohranjivati vrijednosti. Operacija Dodaj dodaje vrijednost u određeni element toga polja, dok operacija Izbriši postavlja vrijednost određenog elementa na 0. Operacija Dodaj nije u mogućnost kreirati novi element unutar već definiranog polja (ne može povećati veličinu polja), a operacija Izbriši nije u stanju izbrisati element u polju (smanjiti veličinu polja) jer bi to narušilo optimiziranosti koda.

Amortizirana cijena operacija	
Operacija	Cijena
Dodaj	log i
Ažuriraj	log i
Suma	log i
Izbriši	log i

8. Zaključak

Fenwickovo stablo je podatkovna struktura koja se koristi za učinkovito izračunavanje zbrojeva prefiksa niza. Također, poznato je pod nazivom binarno indeksirano stablo. Struktura stabla predstavljena je nizom, gdje svaki element u nizu odgovara čvoru u stablu. Fenwickovo stablo pohranjuje informacije na način koji omogućuje pomicanje od nadređenog čvora do čvora njegove djece. Odnos između čvorova roditelja i djece definiran je bitovnom manipulacijom indeksa. Vremenska složenost operacija Dodaj, Ažuriraj, Suma i Izbriši iznosi O(log n), a prostorna složenost, to jest složenost inicijalizacije (konstruktora) jest O(n). Fenwickovo stablo je vrlo učinkovito rješenje za rješavanje problema koji zahtijevaju višestruke operacije upita i ažuriranja nad različitim podskupovima niza. Sve u svemu, Fenwickovo stablo je vrijedna struktura podataka koja se može koristiti za učinkovito rješavanje širokog spektra problema. Njegova sposobnost izvođenja zbrojeva prefiksa, upita za zbrojeve raspona i ažuriranja elemenata čini ga moćnim alatom za programere i istraživače, a njegov jednostavan dizajn čini ga izvrsnim alatom za učenje za razumijevanje principa koji stoje iza dinamičkog programiranja.

Popis literature

[1] D. Kusalić, *Napredno programiranje i algoritmi u C-u i C++-u*, Zagreb: Element d.o.o., 2010.

[2] „Škola koda | Uvod u strukture podataka“, (bez dat.). [Na internetu]. Dostupno: https://skolakoda.github.io/strukture-podataka [pristupano 4.01.2023.].

[3] A. Brajković, „Upiti nad statičkim poljima“, (bez dat.). [Na internetu]. Dostupno: https://materijali.xfer.hr/docs/upiti-nad-intervalima-1/upiti-nad-statickim-poljima/ [pristupano 4.01.2023.].

[4] „Fenwickovo stablo“, (bez dat.). [Na internetu]. Dostupno: https://materijali.xfer.hr/docs/upiti-nad-intervalima-1/fenwickovo-stablo [pristupano 4.01.2023.].

[5] YouTube (Fenwick tree or Binary Indexed Tree), [Na internetu]. Dostupno: https://www.youtube.com/watch?v=CWDQJGaN1gY&t=691s&ab_channel=TusharRoy-CodingMadeSi [pristupano 4.01.2023.].

[6] N. Haliday, R. Jian, „Binary Indexed Trees“, [pdf], 2012., [Na internetu]. Dostupno: https://activities.tjhsst.edu/sct/lectures/1213/bit_09_28_12.pdf [pristupano 15.01.2023.].